

Customizing AI Workflows Cheat Sheet

Irurueta Docs › Guides & References › AI › Customizing AI Workflows
Baseline: VS Code 1.139 with the GitHub Copilot Chat extension (the single extension that now ships every Copilot feature), GitHub Copilot CLI 1.0.88 (@github/copilot) and Claude Code 2.1.283.
Re-verified as current on 2026-09-28. Worked example: ai-catalog (102 iru-* skills, 3 agents) at commit 58fb709. Quick reference only — verify against the official docs before production use.

WHAT TO USE

If you need...	Use
always-on rules	instructions
a reusable procedure	skill or prompt file
an isolated worker	subagent
deterministic enforcement	hook
distribution	plugin
external capability	MCP

SKILL-TESTING CHECKLIST

- Trigger-accuracy tests (should/shouldn't fire)
- Dry runs in a temp checkout
- Golden-output checks
- Regression suites when editing skills
- Versioning and a changelog for skills
- At least three evaluations created
- Tested with Haiku, Sonnet, and Opus
- Tested with real usage scenarios

FILE LOCATIONS

In Claude Code

- ./CLAUDE.md / ~/.claude/CLAUDE.md — project instructions
- ~/.claude/CLAUDE.md — user instructions; ./CLAUDE.local.md — personal (add it to .gitignore)
- .claude/skills/<name>/SKILL.md — project skills; ~/.claude/skills/<name>/SKILL.md — personal
- .claude/agents/<name>.md — project subagents; ~/.claude/agents/<name>.md — user
- .claude/commands/<name>.md — custom commands (older skill format); \$ARGUMENTS, 0-based \$0 / \$1
- .claude/settings.json / .claude/settings.local.json — settings
- .mcp.json — project MCP servers
- .claude-plugin/plugin.json — plugin manifest

In GitHub Copilot (VS Code)

- .github/copilot-instructions.md — repo instructions
- .github/instructions/<name>.instructions.md (applyTo: glob) — path-scoped
- AGENTS.md — open format; Claude Code reads it when no CLAUDE.md is present
- .github/skills/ or .agents/skills/ — project skills (also reads .claude/skills/); ~/.copilot/skills/ / ~/.agents/skills/ — personal (VS Code also ~/.claude/skills/)
- .github/prompts/*.prompt.md — prompt files
- .github/agents/*.agent.md — custom agents
- .vscode/mcp.json — MCP servers

PLUGIN LAYOUT

```
my-plugin/  
├─ .claude-plugin/  
│   └─ plugin.json  
├─ skills/<n>/SKILL.md  
├─ agents/<n>.md  
├─ hooks/hooks.json  
└─ .mcp.json
```

SECURITY CHECKLIST

- Trusting third-party skills, plugins and MCP servers (blast radius differs)
- Prompt injection via tool output and files — treat as data, never instructions
- No hardcoded secrets in hooks and settings
- Least-privilege allowed-tools
- Review plugin contents before install — hooks/MCP run outside the sandbox
- Managed settings for organisations

SKILL.MD & AGENT FRONTMATTER

SKILL.md keys

- name, description, when_to_use
- argument-hint, arguments
- disable-model-invocation, user-invocable
- allowed-tools, disallowed-tools
- model, effort
- context: fork + agent, background
- hooks, paths

- shell, metadata, license, compatibility

Agent .claude/agents/*.md keys

- name, description
- tools, disallowedTools
- model, color, effort
- permissionMode, isolation
- skills, mcpServers, hooks
- maxTurns, memory, background
- omitClaudeMd, initialPrompt, experimental

HOOK EVENTS & EXIT CODES

Can block: PreToolUse, UserPromptSubmit, Stop, SubagentStop, PreCompact.
Observe only: PostToolUse (tool already ran), PostToolUseFailure, SessionStart, SessionEnd, Notification, PostCompact

Exit	Meaning
0	No objection; normal permission flow still applies for PreToolUse; print JSON here for structured control
2	Blocks on events that can block; stderr becomes the reason fed back
other	Non-blocking error; action proceeds; valid JSON on stdout is still read
JSON	PreToolUse: hookSpecificOutput.permissionDecision allow deny ask; Stop: "decision": "block" + reason

PERMISSION RULE SYNTAX

- Bash (mvn *) — any mvn command, including bare mvn
- Read(/secrets/**) — as deny/ask: a secrets dir at any depth; as allow: only <cwd>/secrets
- mcp__<server>__<tool> — one tool of one configured MCP server
- WebFetch(domain:example.com) — exact hostname; domain:*.example.com — subdomains
- Evaluation order: **deny, then ask, then allow** — first match wins
- Modes: default, acceptEdits, plan, auto, dontAsk, bypassPermissions
- VS Code picker: Manual (default), Assisted, Allow all; Autopilot is an agent mode

12 OPTIMISATION TECHNIQUES

1. Context budget awareness
2. Progressive disclosure via reference files and routing tables
3. Context isolation with compact structured summaries
4. Baselines, reporting only regressions
5. Validating once per group
6. Parallel fan-out
7. Model tiering and effort levels
8. Scoped allowed-tools
9. Deterministic work in scripts or hooks rather than prose
10. Idempotency, checkpointing and resumability
11. Avoiding re-prompting (pass-through args)
12. Prompt-cache-friendly structure (stable prefixes)